

WebSocket Complete Flow Cheatsheet

Service · Auto-Reconnect · RxJS Integration · NgRx Effect · Coupa Live-Order Example

Coupa L1 Prep
Sumit · Senior Angular

WebSocket API RxJS Angular 15+ Coupa Pattern

1 WS SERVICE — 2 CONNECT — 3 SEND MSG — 4 RECEIVE — 5 RECONNECT — 6 NGRX EFFECT

WS SERVICE · Injectable, typed, singleton

```
// core/services/ws.service.ts
@Injectable({ providedIn: 'root' })
export class WsService<T> {
  private socket$: WebSocketSubject<T>;
  private msgs$: Observable<T>;
  private url = '';

  connect(url: string): Observable<T> {
    this.url = url;
    this.socket$ = new WebSocketSubject<T>({
      url,
      openObserver: { next: () =>
        console.log('WS connected') },
      closeObserver: { next: () =>
        console.log('WS closed') } },
    );
    this.msgs$ = this.socket$.pipe(
      retry({ count: 5, delay: 3000 }),
      share() // multicast - 1 conn for N subscribers
    );
    return this.msgs$;
  }
}
```

SEND & DISCONNECT · imperative API

```
// Send a typed message to the server
send(msg: T): void {
  this.socket$?.next(msg);
}

// Clean close (sends WS close frame)
disconnect(): void {
  this.socket$?.complete();
}

// Filter stream by message type
on<K extends T>(predicate: (m: T) => m is K)
): Observable<K> {
  return this.msgs$.pipe(filter(predicate));
}
```

AUTO-RECONNECT · exponential back-off pattern

```
// Exponential back-off retry (better than fixed delay)
import { timer } from 'rxjs';

msgs$ = socket$.pipe(
  retryWhen(errors$ =>
    errors$.pipe(
      scan((count, err) => {
        if (count >= 5) throw err;
        return count + 1;
      }, 0),
      delayWhen(count =>
        timer(1000 * 2 ** count)) // 1s, 2s, 4s, 8s, 16s
      ),
    ),
  share()
);
```

COUPA EXAMPLE · Live Order / Approval Feed

```
// Message types matching Coupa's domain
type WsEventType =
  | 'ORDER_APPROVED'
  | 'ORDER_REJECTED'
  | 'BUDGET_EXCEEDED'
  | 'SUPPLIER_UPDATE';

interface WsEvent<P = unknown> {
  type: WsEventType;
  payload: P;
  timestamp: string;
  userId: string;
}

// In component - subscribe to ONE type only
@Component({ changeDetection: OnPush })
export class OrderFeedComponent {
  approvals$ = this.ws
    .on(
      (e): e is WsEvent<Order> =>
        e.type === 'ORDER_APPROVED'
    ).pipe(
      takeUntilDestroyed(this.destroyRef),
      tap(e => this.store.dispatch(
        OrderActions.approved({ order: e.payload })
      ))
    );

  constructor(
    private ws: WsService<WsEvent>,
    private store: Store,
    private destroyRef: DestroyRef
  ) {
    this.ws.connect('wss://api.coupa.com/live');
  }
}
```

NGRX EFFECT · WS messages → Store actions

```
@Injectable()
export class RealtimeEffects {
  // Start WS when app boots, map events → actions
  ws$ = createEffect(() =>
    this.actions$.pipe(
      ofType(AppActions.init),
      switchMap(() =>
        this.ws.connect('wss://...').pipe(
          map(event => {
            switch (event.type) {
              case 'ORDER_APPROVED':
                return OrderActions.approved({ order: event.payload });
              case 'BUDGET_EXCEEDED':
                return BudgetActions.exceeded({ data: event.payload });
              default:
                return AppActions.wsUnknown();
            }
          })
        ),
        catchError(() => of(AppActions.wsError()))
      )
    )
  );

  constructor(
    private actions$: Actions,
    private ws: WsService<WsEvent>
  ) {}
}
```

Interview: WS vs SSE vs Polling

WebSocket Full-duplex · send + receive · best for chat, live feeds, approvals

SSE Server → client only · simpler · good for notifications, logs

Polling HTTP interval · simplest · use when WS infra unavailable

share() Critical — prevents N subscribers from opening N connections

takeUntilDestroyed Always unsubscribe on destroy — WS keeps socket open

1. WebSocket()

Use RxJS WebSocketSubject,
not native WebSocket

2. share()

Multicast the stream so
N components = 1 connection

3. retry()

Always add retry with delay.
Never let WS silently die

4. takeUntil

Unsubscribe on destroy or
the socket stays open forever

5. NgRx Effect

Keep WS in Effect, not
component. Store = truth